

Министерство науки и высшего образования РФ
Федеральное государственное автономное
образовательное учреждение высшего образования
«СИБИРСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт космических и информационных технологий
институт

Кафедра «Программная инженерия»
кафедра

ОТЧЕТ О ПРАКТИЧЕСКОЙ РАБОТЕ

Уровень решения проблемы обработки и генерации естественного языка ч.1
тема

Преподаватель

подпись, дата

А. С. Кузнецов
инициалы, фамилия

Студент

КИ24-04-3М,
номер группы, зачетной книжки

подпись, дата

Н. М. Горовенко
инициалы, фамилия

Красноярск 2025

1 Задача

Необходимо для любого документа или набора документов написать токенизатор, провести нормализацию полученного словаря (стемминг, лемматизация, синонимия, стоп-слова и т. п.) и построить простой анализатор тональности текста на естественном языке (на основе метода VADER-правил или наивного байесовоского классификатора).

2 Ход работы

2.1 Описание исходных данных

В качестве источника рассматриваемых документов выбран [набор данных](#), посвященный анализу тональности текстовых публикаций в социальных сетях.

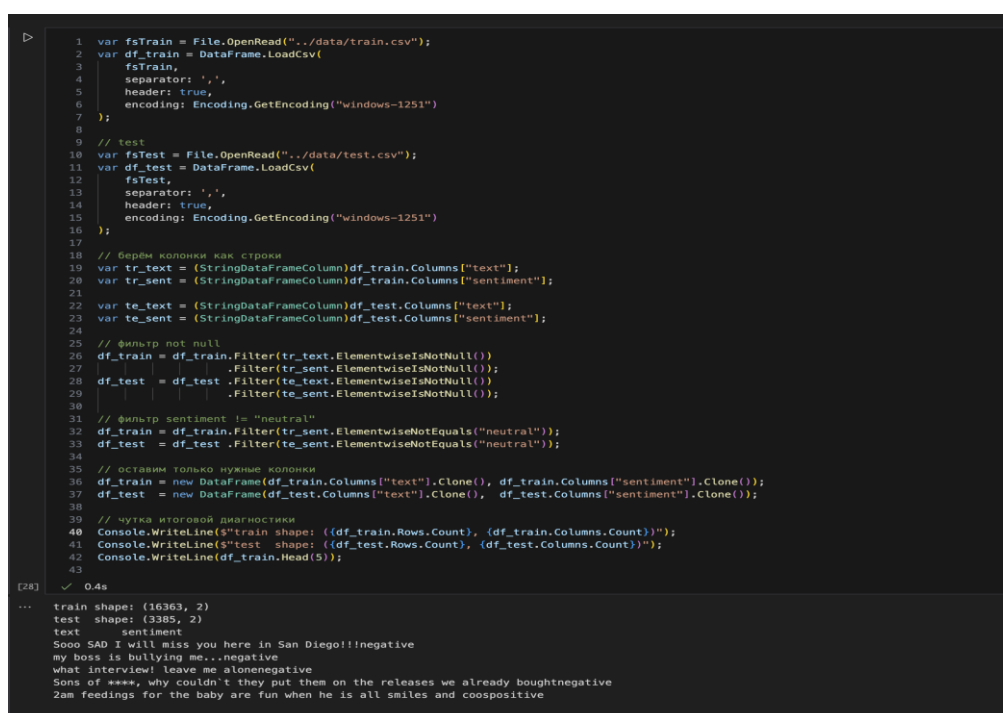
Набор данных изначально разделен на обучающую и тестовую части и состоит из следующих атрибутов:

- textID – идентификационный номер публикации;
- text – текст публикации;
- sentiment – тональность публикации (negative, neutral или positive);
- Time of Tweet – время публикации (morning, noon, night)
- Age of User – возраст пользователя (0-20, 21-30, 31-45, 46-60, 60-70);
- Country – страна проживания пользователя;
- Population -2020 – популяция страны на 2020 год;
- Land Area (KmI) – площадь страны;
- Density (P/KmI) – плотность населения в стране.

Для решения поставленной задачи из набора данных будут использованы только два атрибута: текст публикации и тональность публикации. Также для большей наглядности задача будет сведена к бинарной классификации тональности (позитивная и негативная). Для этого записи, соответствующие нейтральной тональности, будут исключены из анализа.

2.2 Выполнение работы

Выполнение работы начинается с чтения и исследования входных данных, что продемонстрировано на рисунке 1. Из набора данных отбираются только те атрибуты, которые будут использованы для дальнейшего анализа, исключаются записи, соответствующие нейтральной тональности. Затем выполняется вывод размеров датасетов. Как видно, обучающий набор содержит 16363 записи, а тестовый – 3385. В конце рисунка продемонстрированы примеры публикаций и соответствующие метки тональности.



```
1 var fsTrain = File.OpenRead("../data/train.csv");
2 var df_train = DataFrame.LoadCsv(
3     fsTrain,
4     separator: ',',
5     header: true,
6     encoding: Encoding.GetEncoding("windows-1251")
7 );
8
9 // test
10 var fsTest = File.OpenRead("../data/test.csv");
11 var df_test = DataFrame.LoadCsv(
12     fsTest,
13     separator: ',',
14     header: true,
15     encoding: Encoding.GetEncoding("windows-1251")
16 );
17
18 // берём колонки как строки
19 var tr_text = (StringDataFrameColumn)df_train.Columns["text"];
20 var tr_sent = (StringDataFrameColumn)df_train.Columns["sentiment"];
21
22 var te_text = (StringDataFrameColumn)df_test.Columns["text"];
23 var te_sent = (StringDataFrameColumn)df_test.Columns["sentiment"];
24
25 // фильтр not null
26 df_train = df_train.Filter(tr_text.ElementwiseIsNotNull());
27                       .Filter(tr_sent.ElementwiseIsNotNull());
28 df_test = df_test.Filter(te_text.ElementwiseIsNotNull());
29                       .Filter(te_sent.ElementwiseIsNotNull());
30
31 // фильтр sentiment != "neutral"
32 df_train = df_train.Filter(tr_sent.ElementwiseNotEquals("neutral"));
33 df_test = df_test.Filter(te_sent.ElementwiseNotEquals("neutral"));
34
35 // оставим только нужные колонки
36 df_train = new DataFrame(df_train.Columns["text"].Clone(), df_train.Columns["sentiment"].Clone());
37 df_test = new DataFrame(df_test.Columns["text"].Clone(), df_test.Columns["sentiment"].Clone());
38
39 // чутка итоговой диагностики
40 Console.WriteLine($"train shape: {(df_train.Rows.Count), (df_train.Columns.Count)}");
41 Console.WriteLine($"test shape: {(df_test.Rows.Count), (df_test.Columns.Count)}");
42 Console.WriteLine(df_train.Head(5));
43
```

[28] ✓ 0.4s

```
... train shape: (16363, 2)
test shape: (3385, 2)
text sentiment
Sooo SAD I will miss you here in San Diego!!negative
my boss is bullying me...negative
what interview! Leave me alonenegative
Sons of ****, why couldn't they put them on the releases we already boughtnegative
2am feedings for the baby are fun when he is all smiles and coospositive
```

Рисунок 1 – Экранная форма с параметрами мгновенного мониторинга

Для проверки датасета на сбалансированность была построена столбчатая диаграмма, представленная на рисунке 2. Из графика можно сделать вывод, что количество положительных и отрицательных публикаций различается незначительно.

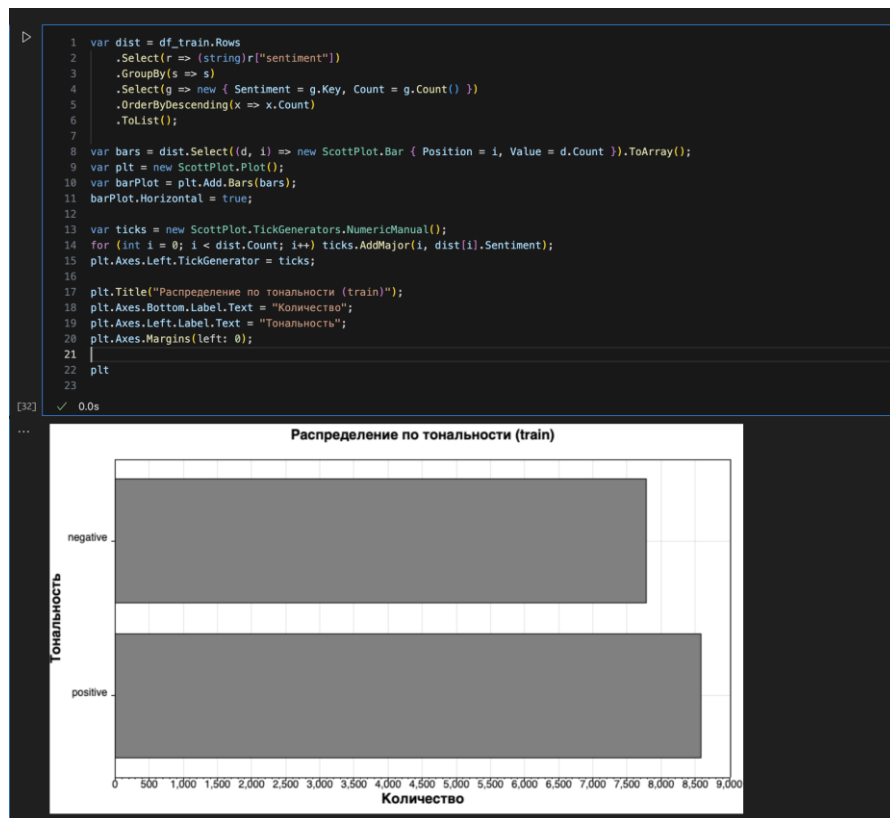


Рисунок 2 – Столбчатая диаграмма распределения целевых меток

На рисунке 3 приведена предобработка данных. Целевая переменная приводится к численному значению с помощью функции «encode_target_label». Текст публикаций подвергается нескольким этапам преобразований, которые сгруппированы в функции «preprocess_test»:

- приведение к нижнему регистру;
- удаление текста в квадратных скобках;
- удаление всех символов, кроме букв и цифр;
- удаление ссылок HTTP/HTTPS;
- удаление HTML-тегов;
- удаление пунктуации;
- удаление переходов на новую строку;
- удаление слов, содержащих цифры;
- токенизация;
- удаление стоп-слов;
- удаление слов, состоящих из одной буквы;
- лемматизация.

```

1 // Небольшой список стоп-слов
2 var stopwords = new HashSet<string>(new []
3 {
4     "a","an","and","are","as","at","be","by","for","from","has","he","in","is","it","its",
5     "of","on","that","the","to","was","will","with","i","you","we","they","me","my","your",
6     "our","their","him","her","us","them","this","these","those","am","are","is","was","were",
7     "been","being","have","has","had","having","do","does","did","doing","would","could",
8     "should","may","might","must","can","shall","ought","need","dare","used"
9 });
10
11 int EncodeTargetLabel(string label) =>
12     label == "positive" ? 1 :
13     label == "negative" ? -1 : 0;
14
15 string SimpleLemmatize(string w)
16 {
17     if (w.EndsWith("ing") && w.Length > 5) return w[..^3];
18     if (w.EndsWith("ed") && w.Length > 4) return w[..^2];
19     if (w.EndsWith("ly") && w.Length > 4) return w[..^2];
20     if (w.EndsWith("s") && w.Length > 3) return w[..^1];
21     return w;
22 }
23
24 List<string> PreprocessText(string text)
25 {
26     text = text.ToLower();
27     text = Regex.Replace(text, @"[\.\*?\\]", "");
28     text = Regex.Replace(text, @"\W", " ");
29     text = Regex.Replace(text, @"https?://\S+|www\.\S+", "");
30     text = Regex.Replace(text, @"<.*?>+", "");
31     text = Regex.Replace(text, @"[\^\w\s]", "");
32     text = Regex.Replace(text, @"\n", "");
33     text = Regex.Replace(text, @"\w*\d\w*", "");
34
35     var words = text.Split(' ', StringSplitOptions.RemoveEmptyEntries)
36         .Where(w => !stopwords.Contains(w))
37         .Where(w => w.Length > 1)
38         .Select(SimpleLemmatize)
39         .ToList();
40     return words;
41 }
42
43 [33] ✓ 0.0s

```

Рисунок 3 – Предобработка данных

После предобработки данных был построен наивный байесовский классификатор для анализа тональности текста, что продемонстрировано на рисунке 4. Полученная модель имеет точность 0.93 на обучающей выборке и 0.87 на тестовой, что говорит о его высокой прогнозной способности.

```

1 public class RowX
2 {
3     public string Text { get; set; }
4     public int Sentiment { get; set; } // 1 / -1
5     public List<string> Tokens { get; set; }
6 }
7
8 List<RowX> ToProcessed(DataFrame df) =>
9     df.Rows
10    .Select(r => new RowX {
11        Text = (string)r["text"],
12        Sentiment = EncodeTargetLabel((string)r["sentiment"]),
13        Tokens = PreprocessText((string)r["text"])
14    })
15    .Where(r => r.Sentiment != 0 && r.Tokens.Count > 0)
16    .ToList();
17
18 var trainX = ToProcessed(df_train);
19 var testX = ToProcessed(df_test);
20
21 Console.WriteLine($"train after preprocess: {trainX.Count}");
22 Console.WriteLine($"test after preprocess: {testX.Count}");

```

[36] ✓ 0.1s

... train after preprocess: 16359
test after preprocess: 2104

Рисунок 4 – Построение и валидация классификатора

На рисунке 5 приведены 10 наиболее информативных признаков, которые учитывает классификатор при принятии решения. Как видно, все эти слова имеют яркую эмоциональную окраску.

```

1 nb.ShowMostInformativeFeatures(10);

```

[55] ✓ 0.1s

... love = positive (score: 0.07)
happy = positive (score: 0.06)
thank = positive (score: 0.06)
good = positive (score: 0.06)
day = positive (score: 0.05)
not = negative (score: 0.04)
sad = negative (score: 0.04)
mother = positive (score: 0.04)
mis = negative (score: 0.03)
great = positive (score: 0.03)

Рисунок 5 – Наиболее информативные признаки

На рисунке 6 можно наблюдать работу классификатора на тестовой выборке.

```
1 for (int i = 0; i < Math.Min(7, testData.Count); i++)
2 {
3     var sample = testData[i];
4     var proba = nb.PredictProba(sample.features);
5     var pred = proba.OrderByDescending(kv => kv.Value).First();
6
7     Console.WriteLine($"original sentence: {sample.text}");
8     Console.WriteLine($"transformed sentence: {string.Join(" ", sample.features.Keys)}");
9     Console.WriteLine($"label: {sample.label}");
10    Console.WriteLine($"model prediction: {(pred.Key == 1 ? "positive" : "negative")}");
11 }
[47] ✓ 0.3s
... original sentence: Shanghai is also really exciting (precisely -- skyscrapers galore). Good tweeps in China: (SH) (BJ).
transformed sentence: shanghai also real excit precise skyscraper galore good tweep china sh bj
label: 1
model prediction: positive
original sentence: Recession hit Veronique Branquinho, she has to quit her company, such a shame!
transformed sentence: recession hit veronique branquinho she quit company such shame
label: -1
model prediction: negative
original sentence: happy bday!
transformed sentence: happy bday
label: 1
model prediction: positive
original sentence: http://twitpic.com/4w75p - I like it!!
transformed sentence: http twitpic com like
label: 1
model prediction: positive
original sentence: that's great!! weee!! visitors!
transformed sentence: great weee visitor
label: 1
model prediction: positive
original sentence: I THINK EVERYONE HATES ME ON HERE lol
transformed sentence: think everyone hate here lol
label: -1
model prediction: negative
original sentence: soooooo wish i could, but im in school and myspace is completely blocked
transformed sentence: soooooo wish but im school myspace complete block
label: -1
model prediction: negative
```

Рисунок 6 – Пример работы классификатора

3 Вывод

Для выбранного набора документов был написан токенизатор, выполнена лемматизация, удалены стоп слова и другие элементы, которые не имеют семантической важности. На основе предобработанного текста построен анализатор тональности текста на естественном языке на основе наивного байесовского классификатора.